56262

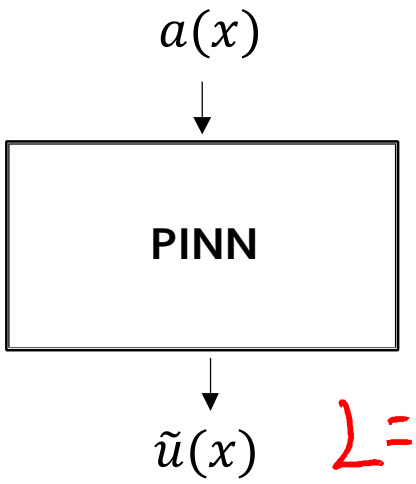
Neural Operator 3

DeepONet: Learning nonlinear operators for identifying differential equations based on the universal approximation theorem of operators,
Nature Machine Intelligence (2021)

PINN vs. Neural Operator

PDE(u) = f(x)
a,x

PINN

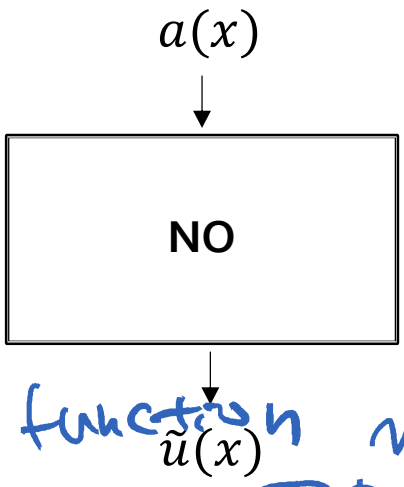


PDE(u)
a,x
=
$L = \|f(x)\|^2$

$$\mathcal{L} = \|\mathcal{N}[u_\theta](x)\|^2$$

- Minimizes PDE residual
  - ✓ Just individual solutions

Neural Operator



(implicit physics)

Continuous function mapping

$$G : a(x) \rightarrow u(x)$$

- Maps input function to solution function
  - ✓ Various initial conditions
  - ✓ Different boundary conditions
  - ✓ Different coefficients
  - ✓ Various grid resolution

Application of Neural Operators

- **A central application** of neural operators
  - Learning solution operators defined by parametric partial differential equations (PDE)
  - All problems follow this general structure:

$$G^\dagger : \text{input function} \to \text{solution function}$$

    - coefficient $a(x) \to$ solution $u(x)$
    - initial condition $u_0(x) \to$ solution $u(x,t)$

- **General PDE Setup**
  - Considering PDEs of the form:

$$L_a u = f$$

  - Aiming to learn mappings like:
    - $a \mapsto u$
    - $f \mapsto u$

Test Problems 1/4

- **Poisson Equation (Simple, analytical case)**

$$-\frac{d^2}{dx^2}u(x) = f(x)$$

- Input: $f(x)$
- Output: $u(x)$

f → u

- **Applications**
  - Given a source (e.g., charge or heat) $f(x)$, how does its effect spread $u(x)$ in space?
    - Electrostatics (electric potential from charge distribution)
    - Gravitational potential fields
    - Steady-state heat distribution
    - Image processing (Poisson image editing)

Test Problems 2/4

- **Darcy Flow (Parametric PDE)**

$$-\nabla \cdot (a(x)\nabla u(x)) = f(x)$$

  - Input: coefficient field $a(x)$
  - Output: solution $u(x)$

$a \to u$

- **Applications**
  - How does fluid flow (e.g., potential) $u(x)$ through a material whose properties $a(x)$ vary in space?
    - Groundwater flow modeling
    - Oil and gas reservoir simulation
    - Fluid flow in porous media
    - Environmental engineering (pollutant transport in soil)

Test Problems 3/4

- **Burgers' Equation (Time-dependent nonlinear PDE)**

$$\partial_t u + u\partial_x u = \nu \partial_{xx} u$$

  - Input: initial condition $u_0(x)$
  - Output: $u(x, t = 1)$

- **Applications**
  - A flow $u(x)$ that starts from an initial condition $u_0(x)$, and evolves over time by transporting and diffusing
    - Shock wave formation
    - Traffic flow modeling
    - Simplified fluid dynamics
    - Toy model for turbulence

Test Problems 4/4

Problem	Input	Varying elements	Fixed elements
Poisson	$f(x)$ (source function)	$f(x)$ (forcing function)	**PDE structure (operator)** Boundary conditions
Darcy Flow	$a(x)$ (coefficient field)	$a(x)$ (material/permeability)	**PDE structure (operator)** Source function $f(x)$
Burgers' Equation	$u_0(x)$ (initial condition)	$u_0(x)$	**PDE structure (operator)** PDE coefficient $a(x)$

Disadvantages of Neural Operator

	PINN	Neural Operator
Data requirement	Low (Physics-based loss)	High (Data-driven)
Interpretability	Low (Black Box)	Low (Implicit Physical Law)

1. **High Data Requirement**
   - Neural Operators typically require a large amount of training data.
2. **Limited Interpretability**
   - Neural Operators, like many deep learning models, suffer from low interpretability.

DeepONet 1/4

- **DeepONet : Approximate a nonlinear operator**
  - Map input function to output (solution) function

$$u(x) \rightarrow G(u)(y)$$

Output location

Input function

Output function

- **Partial Interpretability :** Representation in Function Space
  - The output a continuous function represented as a linear combination (composition) of basis functions.

$$G(u)(y) = \sum_{k=1}^{p} b_k(u)\, t_k(y)$$

- $t_k(y)$: basis functions
- $b_k(u)$: coefficients depending on the input function

DeepONet 2/4

- **Partial Interpretability:** How to Incorporate Boundary Conditions and Physics into DeepONet

$$G(u)(y) = \sum_{k=1}^{p} b_k(u)\, t_k(y)$$

- $t_k(y)$: basis functions
- $b_k(u)$: coefficients depending on the input function

- Instead of using arbitrary basis functions $t_k(x)$, we design them to satisfy boundary conditions
- **Example:** Dirichlet Boundary Condition

Given:

$$u(0) = 0, \quad u(1) = 0$$

Define basis functions as:

$$t_k(x) = x(1-x)\phi_k(x)$$

DeepONet 3/4

- **Partial Interpretability:** How to Incorporate Boundary Conditions Physics into DeepONet

$$G(u)(y) = \sum_{k=1}^{p} b_k(u)\, t_k(y)$$

- $t_k(y)$: basis functions
- $b_k(u)$: coefficients depending on the input function

- By designing the trunk input or structure, we can encode physical properties of the solution
- **Example:** Fourier Features

$$x \to \sin(kx), \cos(kx)$$

- $t_k(y)$: a spatial pattern
- $b_k(u)$: its importance for a given input

$t = \sin(kx)$

$t = \cos(kx)$

DeepONet 4/4

- **Strong Generalization:** Input-Dependent Adaptation
  - Because the basis functions are shared
  - The coefficients $b_k(u)$ change depending on the input function.

Varying coefficient according input data

$$G(u)(y) = \sum_{k=1}^{p} b_k(u)\, t_k(y)$$

basis functions

- Only the coefficients change
- The representation structure remains the same
- This enables function-level generalization

- **Theoretical Guarantee:** Universal Approximation Theorem
  - With enough basis functions, DeepONet can approximate any continuous operator

Architecture 1/2

1. **Branch Network**
   - Input: sampled values of function u

$$[u(x_1), u(x_2), \ldots, u(x_m)]$$

   - Goal : encode the input coefficient $b_k(u)$

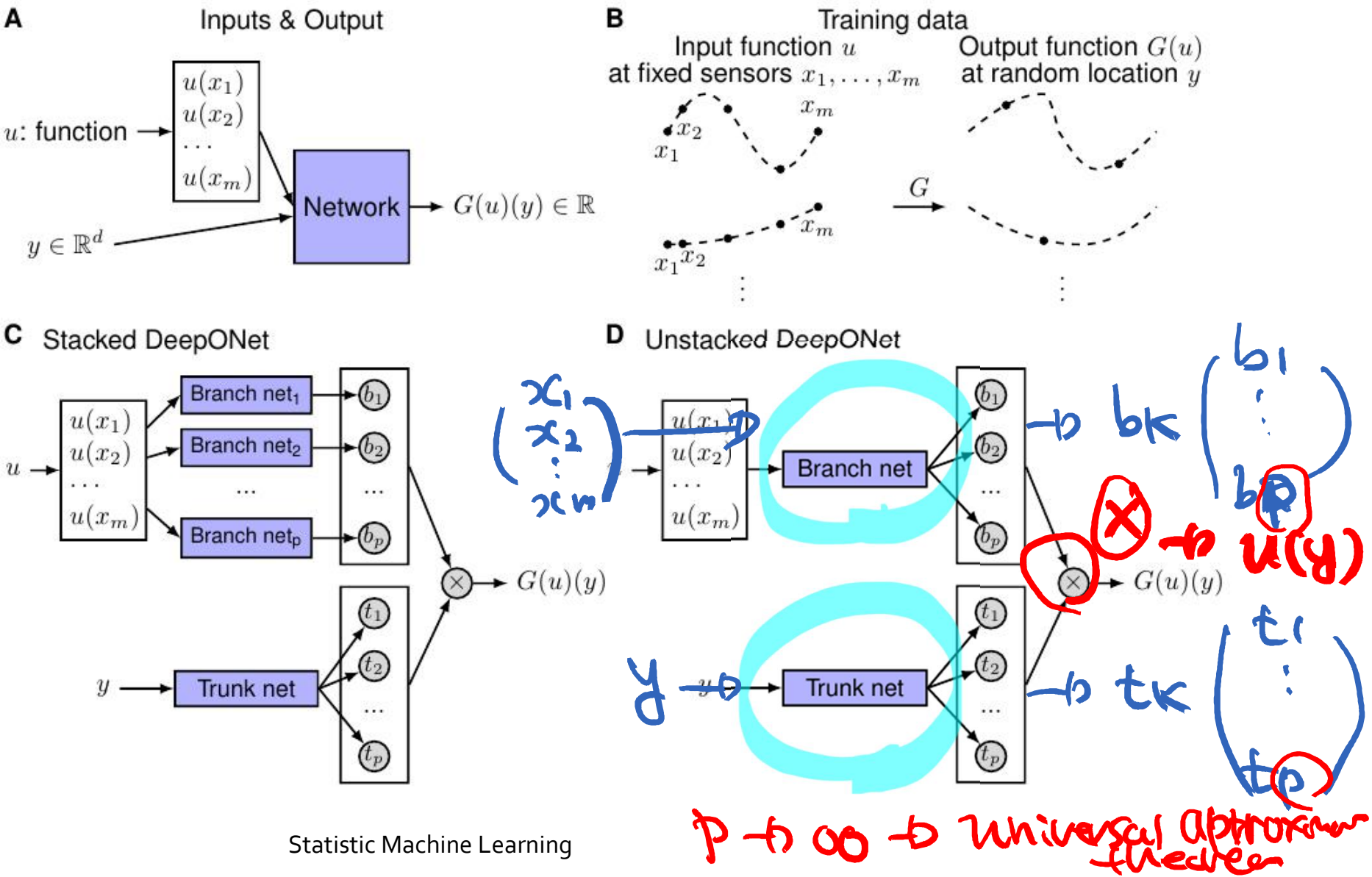
2. **Trunk Network**
   - Input: output location y
   - Goal : encode basis function $t_k(y)$ where we evaluate the output

- **Final Output**

$$G(u)(y) \approx \sum_{k=1}^{p} b_k \cdot t_k$$

- branch output b_k
- trunk output t_k

Architecture 2/2



Training Process 1/2

- **Objective**
  - The goal of DeepONet is to approximate the operator: $G(u)(y)$
  - by learning a parametric model: $G_\theta(u)(y) \approx G(u)(y)$

$u_\theta(y) \approx u(y)$

- **Training Data**
  - The training dataset consists of samples of the form:

$$\{(u^{(i)}, y^{(j)}, G(u^{(i)})(y^{(j)}))\}$$

Input , u GT

- $u^{(i)}$: input function (represented by sensor values)
- $y^{(j)}$: evaluation location
- $G(u^{(i)})(y^{(j)})$: ground-truth output (solution)

Training Process 2/2

- **Training Method**
  - DeepONet is trained using supervised learning.

$$L = \frac{1}{N} \sum_{i,j} \left| G(u^{(i)})(y^{(j)}) - G_\theta(u^{(i)})(y^{(j)}) \right|^2$$

- **Training Procedure**
  1. Sample an input function $u(x)$ and evaluate it at sensor points
  2. Pass the sampled values through the **branch network**
  3. Input location y into the **trunk network**
  4. Combine both outputs via inner product
  5. Compute loss and update parameters via backpropagation

Physics Informed DeepONet(PIDoN)

- **PINN + DeepONet**
  - Low data requirement (Strong inductive bias)
  - High Interpretability (Compositions of basis function)

$$L = L_{\text{data}} + L_{\text{physics}}$$

